다음 작업 인계 문서

브랜치: `task-interpretation-agent` | 최종 작업일: 2026-06-04

---

1. 현재까지 완료된 작업

M2 (sub2_new/dev2/)

작업	상태	관련 파일
keyword/sub_category 컬럼명 변경	✓ 완료	`schema.py`, `concept_mapper.py`, `llm_judge.py`, `store.py`, `test_concept_mapper.py`
M4 연동 필드 추가 (code_context, taxonomy)	✓ 완료	`schema.py`, `concept_mapper.py`
scan_issues 컬럼 정의 문서	✓ 완료	`pj3_data/SCAN_ISSUES_COLUMNS.md`

pj3_data (평가 데이터셋)

작업	상태	관련 파일
M2 mixed 케이스 6개	✓ 완료	`pj3_data/datasets/m2_custom_security_cases/mixed_case_01~06/`
M3 mixed 케이스 6개	✓ 완료	`pj3_data/datasets/m3_mixed_project_cases/mixed_case_001~006/`
DB 스키마 문서 (DDL)	✓ 완료	`pj3_data/DB_SCHEMA.md`
scan_issues 컬럼 정의	✓ 완료	`pj3_data/SCAN_ISSUES_COLUMNS.md`
총 케이스 수	✓ 84개	`pj3_data/datasets/dataset_manifest.json`

---

2. 다음에 해야 할 작업 목록

우선순위 1 — M3 컬럼 추가 (하루 작업)

M3 결과 적재 시 M4가 가져갈 컬럼 5개가 아직 미구현 상태.

대상 파일:

- `test-performance-agent/contracts.py` — Issue 모델에 필드 추가
- `test-performance-agent/persist.py` — INSERT 쿼리에 컬럼 추가

추가해야 할 컬럼:

```
# contracts.py Issue 모델에 추가
keyword: str = ""              # rule_id → keyword 변환 (PERF-002 → "N+1
```

```
sub_keyword: str = ""           # 탐지된 실제 함수명 (예: "load_cart_items"
metric_value: str = ""          # 구체적 수치 (예: "15회", "45.5%", "O(N²)"
sub_category: str = ""          # 커리큘럼 트리 매핑 코드 (예: "Database/API
code_context_summary: str = "" # LLM 한 줄 설명
```

keyword 변환 테이블 (`pj3_data/SCAN_ISSUES_COLUMNS.md` §4 참고):

```
PERF-002 → N+1 쿼리
PERF-001 → 시간복잡도
PERF-003 → 메모리 누수
TEST-001 → 커버리지 미충족
TEST-003 → 엣지케이스 누락
TEST-004 → 테스트 누락
```

우선순위 **2** — 하네스 엔지니어링 + 클린 아키텍처 적용

`CLAUDE.md §3, §4`에 설계는 완료됨. 실제 구현 필요.

A. LangSmith 연동 확인

```
# .env에 아래 추가 후 실제 에이전트 실행
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=ls__xxxx
LANGCHAIN_PROJECT=dev2-<본인github_id>
```

- `sub2_new/dev2/` 실행 시 LangSmith 대시보드에 trace 뜨는지 확인

B. AgentHarness 구현 확인

- `common/harness.py` 존재 여부 확인
- 없으면 `CLAUDE.md §6` 패턴 기준으로 구현 필요
- `run_parallel_agents()`, `make_timeout_result()` 함수 구현

C. Repository 패턴 점검

- `sub2_new/dev2/db.py` 읽고 SQL이 직접 노출됐는지 확인
- `CLAUDE.md §2` 기준: 에이전트 코드에 SQL 직접 노출 금지

작업 시작 명령어:

`sub2_new/dev2/db.py 읽어보고 Repository 패턴 적용 가능한지 분석해줘`

우선순위 **3** — **Google Cloud SQL** 적재 테스트

`pj3_data/DB_SCHEMA.md` DDL로 실제 테이블 생성 후 M2/M3 결과 적재 테스트.

순서:

1. Cloud SQL 인스턴스에 `DB_SCHEMA.md` DDL 실행
2. `sub2_new/dev2/db.py`의 insert 함수가 신규 컬럼 (keyword, sub_category 등) 포함하는지 확인

3. M2 sqli_case 하나 돌려서 scan_issues 테이블에 적재 확인

---

3. 주요 파일 위치 (빠른 참조)

```
Task-Link/
├── sub2_new/dev2/              ← M2 작업 폴더
│   ├── schema.py               ← Issue 모델 (keyword, sub_category 포함)
│   ├── nodes/
│   │   ├── concept_mapper.py   ← M4 매핑 로직
│   │   ├── llm_judge.py        ← LLM severity 재판단
│   │   └── store.py            ← PostgreSQL 적재
│   └── tests/
│       └── test_concept_mapper.py
├── test-performance-agent/     ← M3 작업 폴더
│   ├── contracts.py            ← M3 Issue 모델 (keyword 등 미추가)
│   └── persist.py              ← M3 적재 (8개 컬럼만)
└── pj3_data/                   ← 평가 데이터셋
    ├── DB_SCHEMA.md            ← 6개 테이블 DDL
    ├── SCAN_ISSUES_COLUMNS.md  ← scan_issues 컬럼 전체 정의
    └── datasets/               ← 84개 케이스
```

---

4. 현재 PR 상태

PR	브랜치	base	상태
#17	task-interpretation-agent	dev	🟡 오픈 (머지 대기)

---

5. 다음 세션 시작 명령어 예시

```
# M3 컬럼 추가
test-performance-agent/contracts.py 읽고 M4용 컬럼 5개 추가해줘 (keyword, s

# 하네스 엔지니어링 분석
sub2_new/dev2/db.py 읽어보고 하네스 엔지니어링 + 클린 아키텍처 적용 가능한지 분

# PR 머지
PR #17 머지해줘
```